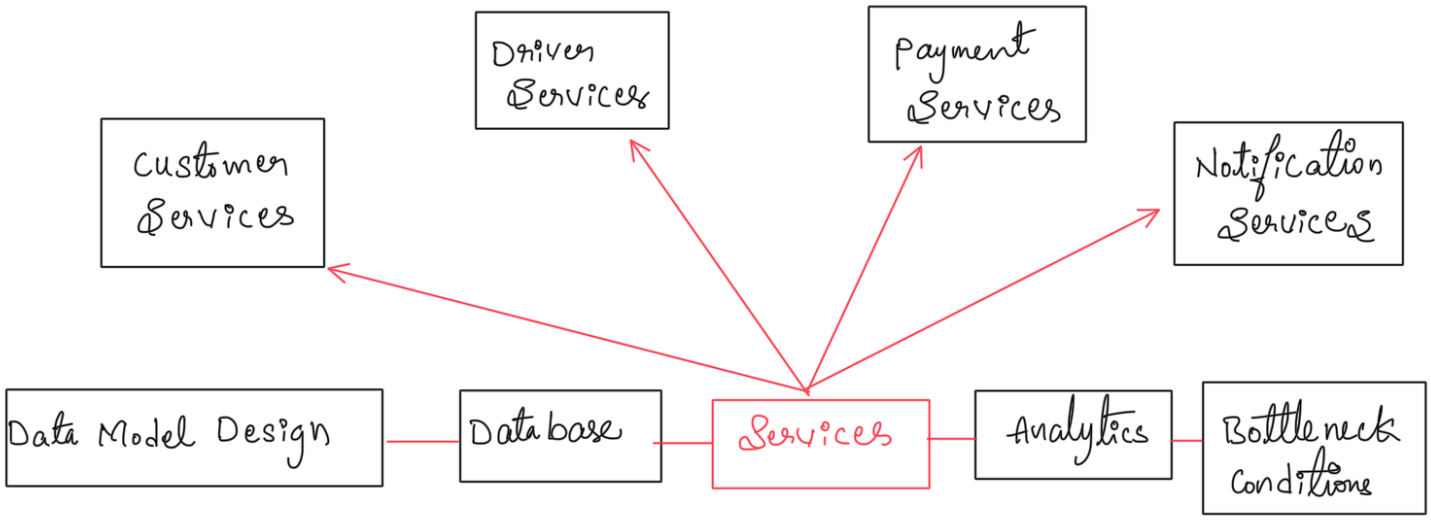
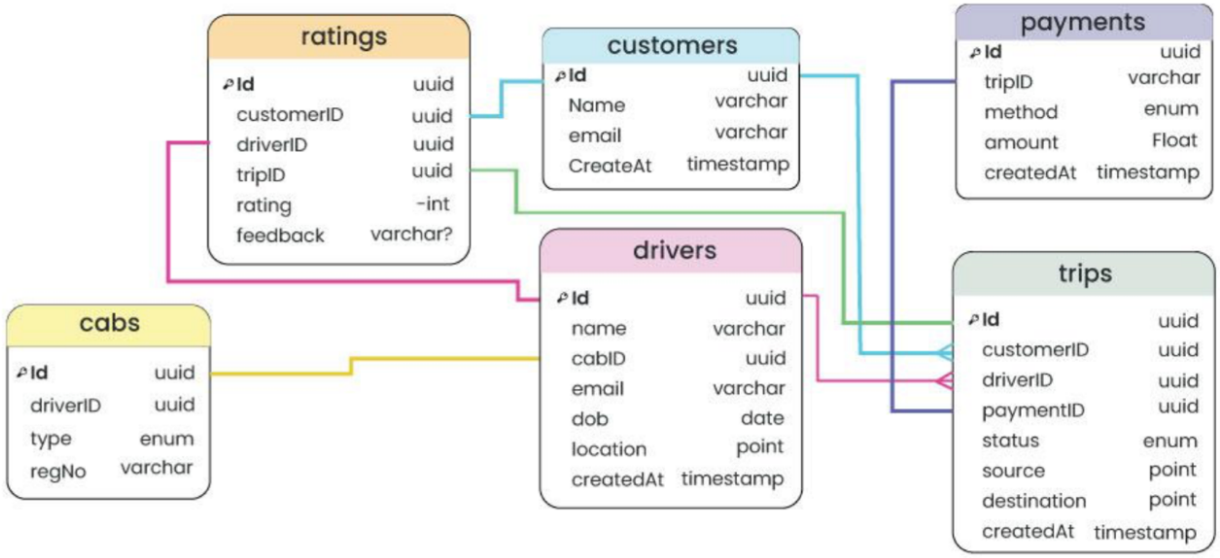
Exploring System Design of UBER part - 3

DAY 115
02/11/2025

High Level Design of Uber App



Data Model Design



Data Bases

Uber had to consider some of the requirements for the database for a better customer experience. These requirements are...

- The database should be horizontally scalable. You can linearly add capacity by adding more servers.
- It should be able to handle a lot of reads and writes because once every 4-second cabs will be sending the GPS location and that location will be updated in the database.
- The system should never give downtime for any operation. It should be highly available no matter what operation you perform (expanding storage, backup, when new nodes are added, etc).

Earlier Uber was using the RDBMS PostgreSQL database but due to scalability issues uber switched to various databases. Uber uses a NoSQL database (schemaless) built on top of the MySQL database.

- Redis for both caching and queuing. Some are behind Twemproxy (which provides scalability of the caching layer). Some are behind a custom clustering system.
- Uber uses Schemaless (built in-house on top of MySQL), Riak, and Cassandra. Schemaless is for long-term data storage. Riak and Cassandra meet high-availability, low-latency demands.
- MySQL database.
- Uber is building their own distributed column store that's orchestrating a bunch of MySQL instances.

Services

- **Customer Service:** This service handles concerns related to customers such as customer information and authentication.
- **Driver Service:** This service handles driver-related concerns such as authentication and driver information.
- **Payment Service**: This service will be responsible for handling payments in our system.
- **Notification Service**: This service will simply send push notifications to the users. It will be discussed in detail separately.

Analytics

To optimize the system, minimize the cost of the operation and for better customer experience uber does log collection and analysis. Uber uses different tools and frameworks for analytics. For log analysis, Uber uses multiple Kafka clusters.

Kafka takes historical data along with real-time data. Data is archived into Hadoop before it expires from Kafka. The data is also indexed into an Elastic search stack for searching and visualizations. Elastic search does some log analysis using Kibana/Graphana. Some of the analyses performed by Uber using different tools and frameworks are...

- Track HTTP APIs
- Manage profile
- Collect feedback and ratings
- Promotion and coupons etc
- Fraud detection
- Payment fraud
- Incentive abuse by a driver
- Compromised accounts by hackers. Uber uses historical data of the customer and some machine learning techniques to tackle this problem.

How to Handle the Data Centes Failure?

Datacenter failure doesn't happen very often but Uber still maintains a backup data center to run the trip smoothly. This data center includes all the components but Uber never copies the existing data into the backup data center.

Then how does Uber tackle the data center failure??

- It actually uses driver phones as a source of trip data to tackle the problem of data center failure.
- When The driver's phone app communicates with the dispatch system or the API call is happening between them, the dispatch system sends the encrypted state digest (to keep track of the latest information/data) to the driver's phone app.
- Every time this state digest will be received by the driver's phone app. In case of a data center failure, the backup data center (backup DISCO) doesn't know anything about the trip so it will ask for the state digest from the driver's phone app and it will update itself with the state digest information received by the driver's phone app. Untitled Diagram

Internal Team Testing Suggestions

To Ensure Uber's complex System runs Smoothly and reliably, the internal teams must perform through testing across multiple areas.

1. Functional Testing

- Verify all user flows like ride booking, driver tracking, cancellations, and payments.
- Test edge cases such as no cabs available or last-minute cancellations.

2. Load and Performance Testing

- Simulate millions of daily requests to measure dispatch latency and Kafka throughput.
- Test database scalability for frequent GPS location updates and trip data.

3. Integration Testing

- Validate communication between Supply, Demand, Dispatch, Payment, and Notification services.
- Test WebSocket connections for real-time driver and rider updates.
- Verify Kafka event streaming and data pipeline flows correctly.

4. Fault Tolerance and Recovery Testing

- Simulate data center failure to check backup data center recovery using state digests from driver phones.
- Test system behavior under network partitions and node failures.
- Ensure data consistency and recovery after failover.

5. Geospatial and Dispatch Logic Testing

- Validate location mapping using S2 library cells for accurate supply-demand matching.
- Test dispatch optimization algorithms and ETA calculations using road network graphs and OSRM.

6. Security Testing

- Ensure secure authentication and authorization for drivers and riders.
- Test encryption of sensitive data, including state digests shared between apps and servers.
- Conduct penetration testing on APIs and WebSocket communication.

7. Analytics Pipeline Testing

- Verify Kafka data archiving, Hadoop storage, and ElasticSearch indexing.
- Test analytics dashboards for accurate real-time and historical insights.

8. Scalability Testing

- Dynamically add or remove Ringpop nodes and ensure load is balanced correctly.
- Test consistent hashing and shard allocation for distributed dispatch